

PROJECT EXPLANATION

Revon Architecture

How the system works, what the demo proves, and how to explain it confidently to your teacher.

The idea in one sentence: Revon is a Git-like versioned key-value database that stores each tree node by its content hash, allowing unchanged subtrees to be shared between versions and skipped during comparison.

The problem it solves

- Full-snapshot versioning copies an entire table even when only a few records changed.
- Naive diffing scans every key even when the actual change is tiny.
- Revon uses hashes to identify identical data, reuse it, and avoid unnecessary traversal.

The 30-second explanation

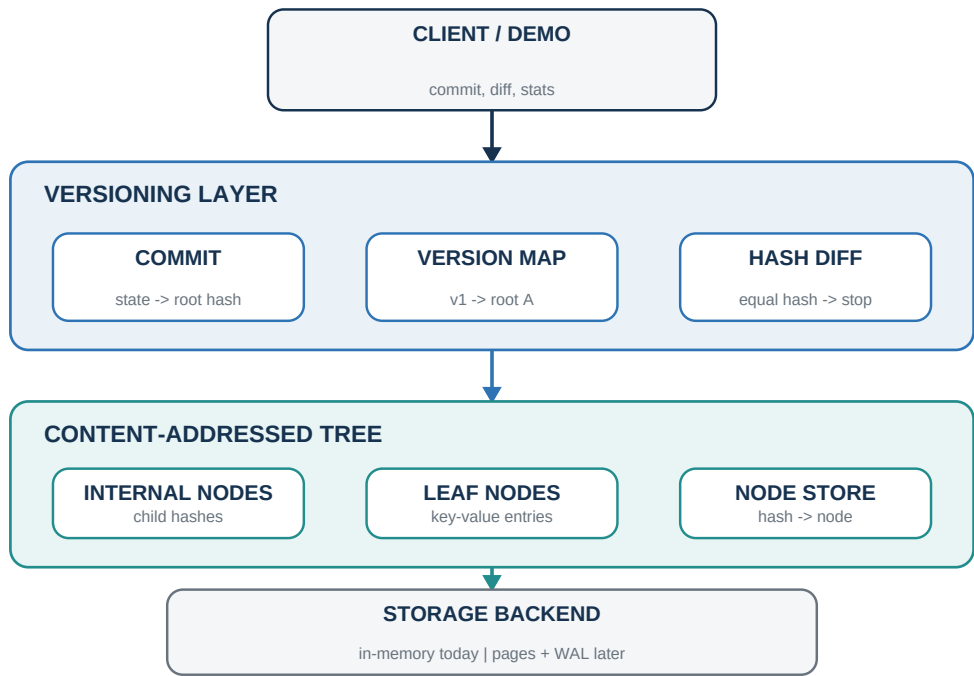
Say this: Our prototype stores the database as a hash-based tree. Every node receives a SHA-256 hash from its contents. If two versions contain the same subtree, that subtree has the same hash and is reused instead of copied. During diff, matching hashes prove that the complete subtree is identical, so the algorithm stops there and explores only changed paths.

What exists today

A Python, in-memory proof of concept with commit, version roots, hash-guided diff, reuse statistics, automated tests, and a three-model benchmark. Data is not yet durable after the program exits.

1. System architecture

The design is easiest to understand as four layers. The first three are implemented in memory; the durable storage backend is planned.



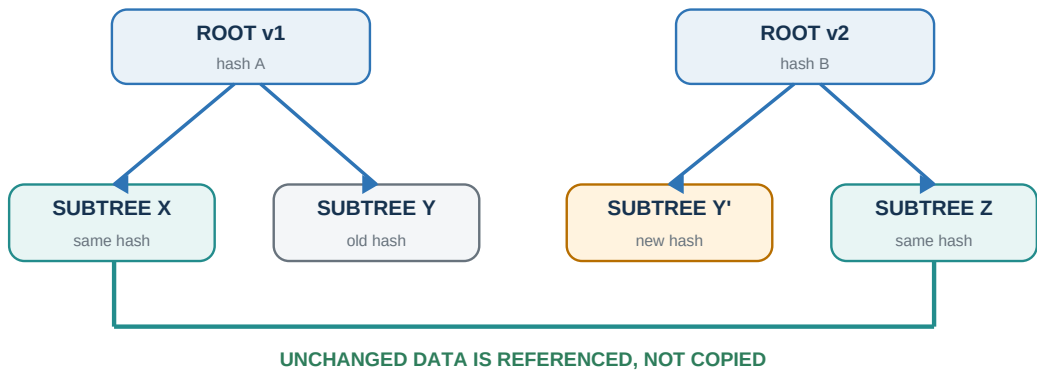
Layer	Responsibility	Current status
Client / demo	Creates data, changes keys, commits versions, requests diff, and prints statistics.	Implemented
Versioning	Records root hashes, computes commits, walks roots during diff, and reports reuse.	Implemented
Hash tree	Routes keys deterministically, stores key-value leaves, and hashes internal child references.	Implemented
Storage backend	Persists nodes through pages, buffer pool, WAL, recovery, and transactions.	Future work

Important internal structures

Structure	Meaning
node_store[hash]	A content-addressed map from a node's SHA-256 ID to its immutable content.
versions[number]	Maps each version number to the root hash representing its complete state.
Internal node	Contains numbered child slots whose values are child hashes.
Leaf node	Contains the actual key-value entries for one route bucket.

2. Commit and structural sharing

A commit does not save a command. It records a new root hash. That root points through hashed internal nodes to the complete key-value state.



1	Route each key SHA-256 of the key determines the fixed child slots followed through the tree.
2	Build leaves Keys that reach the same route bucket are stored together in a deterministic leaf.
3	Hash bottom-up Each leaf is hashed; parent hashes are then computed from their child hashes until the root is reached.
4	Intern each node If a node hash already exists, Revon reuses that node. Otherwise it stores a new node.
5	Record the version The new version number is mapped to the new root hash.

Why three changed keys create more than three new hashes: Each changed leaf receives a new hash, and every ancestor on its path to the root must also receive a new hash. In the demo, three record changes create 12 new tree nodes while 1,402 nodes are reused.

Complexity today vs. intended design

Operation	Current prototype	Intended incremental version
Commit	Receives and rebuilds the complete candidate state: approximately $O(N)$.	put/delete copies only one path: approximately $O(\text{tree depth})$.
Diff	Descends only through mismatching hashes.	Same hash-pruned behavior.
Storage	Unique nodes are retained in RAM.	Unique nodes are persisted on pages.

3. How diff works

Central rule: If the two node hashes are equal, stop. Equal content hashes mean the complete subtrees are identical, so there is no reason to inspect their children.

1	Compare roots If the root hashes match, both versions are identical and diff finishes immediately.
2	Compare child slots When roots differ, the algorithm compares corresponding child hashes.
3	Prune matching subtrees Every equal pair is counted as skipped and is not opened.
4	Reach changed leaves Only differing paths reach leaf-level key comparison.
5	Return changed keys Added, removed, or updated keys form the final result.

Demo result and meaning

Printed result	What it proves
v1: 1,414 new nodes	The initial version constructs the complete tree.
v2: 12 new, 1,402 reused	Only changed paths require new nodes.
99.2% shared	Most structure is referenced by both versions.
3 changed keys	The diff returns exactly the intended mutations.
54 pairs compared, 42 skipped	Hash equality prevents unnecessary recursive work.

Benchmark interpretation

At 100,000 rows, the measured prototype diff took about 1.1 ms versus about 225 ms for a full-state scan. Revon used about 3 MiB of logical serialized storage across five versions versus about 10 MiB for full copies. The operation log was faster for the short 40-operation history, which is expected.

These are prototype measurements on one machine, not universal performance guarantees.

4. How to explain it to your teacher

Use this order: problem, idea, architecture, demo evidence, limitation, next step. It keeps the explanation accurate and easy to follow.

Presentation script: Ma'am, this is an in-memory versioned key-value database. Data is organized as a tree, and every node has a SHA-256 content hash. Each version is represented by one root hash. When a few keys change, unchanged subtrees keep their hashes and are reused. Diff starts at the roots and stops at matching hashes. In our 1,000-key demo, three changes created 12 new nodes, reused 1,402 nodes, and produced 99.2 percent sharing. Today, commit still rebuilds the complete candidate state and data is not persisted after exit. Next we will add incremental updates, then disk storage and recovery.

Likely viva questions

Question	Short answer
Is this Git?	No. It borrows Git's content-addressing and root-based version idea, but applies it to structured key-value records.
Does it store commands?	No. The current prototype stores hashed nodes and version root hashes in RAM. It does not persist terminal commands.
Why SHA-256?	It gives deterministic content IDs with an extremely low practical collision probability and makes identical nodes easy to detect.
Why are 12 nodes new for 3 changes?	The changed leaves and their parent paths up to the root all receive new hashes.
Why is diff fast?	Matching hashes summarize complete subtrees, allowing the traversal to stop early.
What is missing?	Incremental updates, disk pages, WAL, crash recovery, transactions, checkout, branching, and merge.
Is Revon state or operational?	It is currently a content-addressed state model. The operation log in benchmarks is a separate baseline.

What not to claim

- Do not say data is permanently stored; the current node store is in memory.
- Do not say commit is already $O(\text{changes})$; the current commit still receives the complete state.
- Do not call Revon an operation-log hybrid; that model is currently only a benchmark baseline.
- Do not describe disk pages, WAL, branching, or merge as implemented features.

Strong closing line: The demo proves two things today: unchanged data is not duplicated in the node store, and version comparison skips unchanged subtrees using only their hashes.